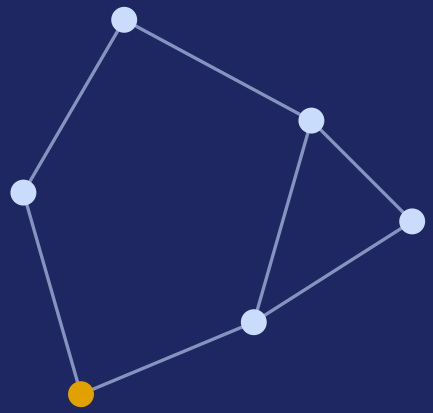
ARTIFICIAL INTELLIGENCE



Lecture 02: Searching — Introduction

From "what is intelligence" to the first formal notion of an agent — now with worked examples

Dr. Diaa M. Fayed

Computer Science and AI

CS 5/7320
Artificial Intelligence

Solving problems by searching

AIMA Chapter 3

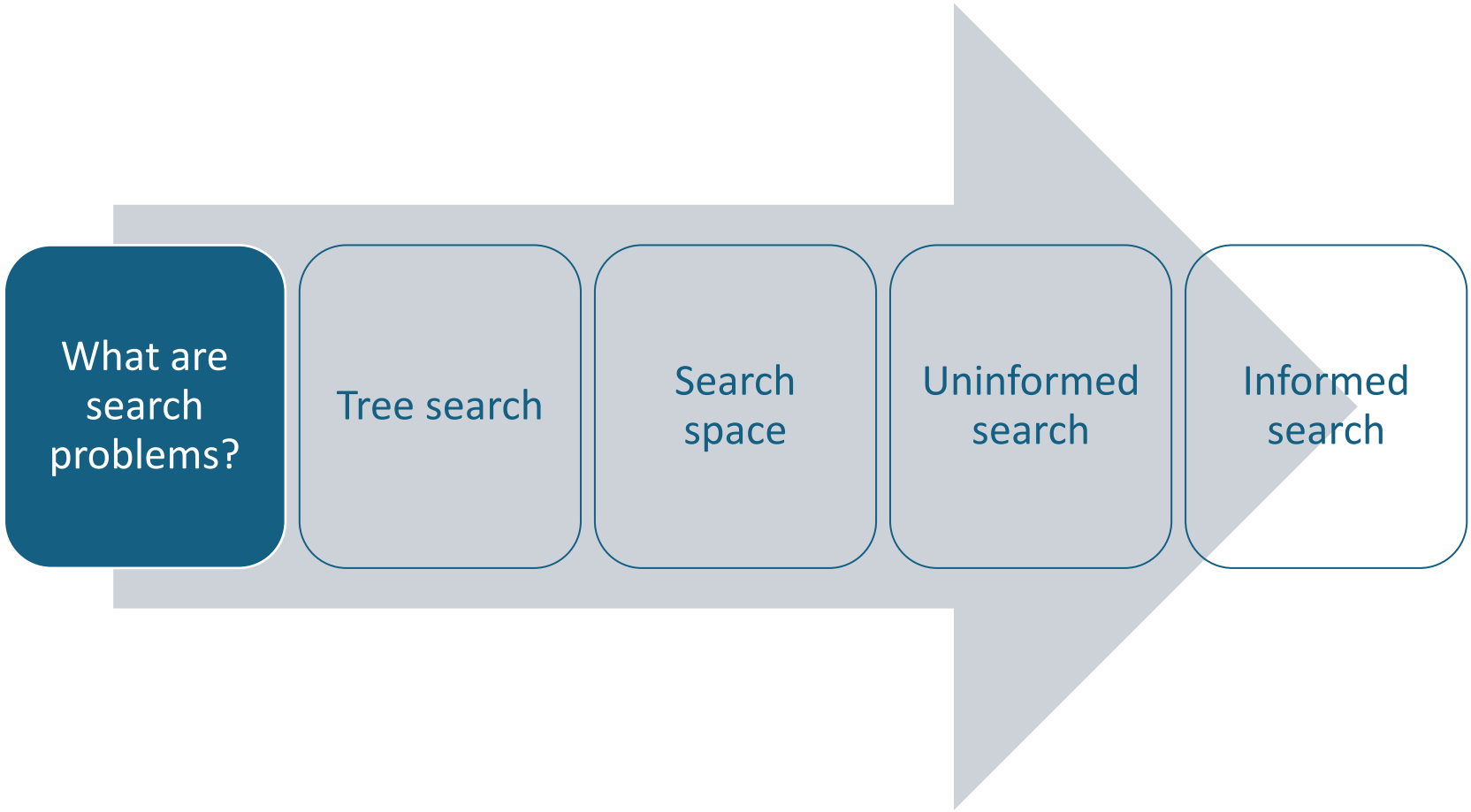
Slides by Michael Hahsler
based on slides by Svetlana Lazepnik
with figures from the AIMA textbook.

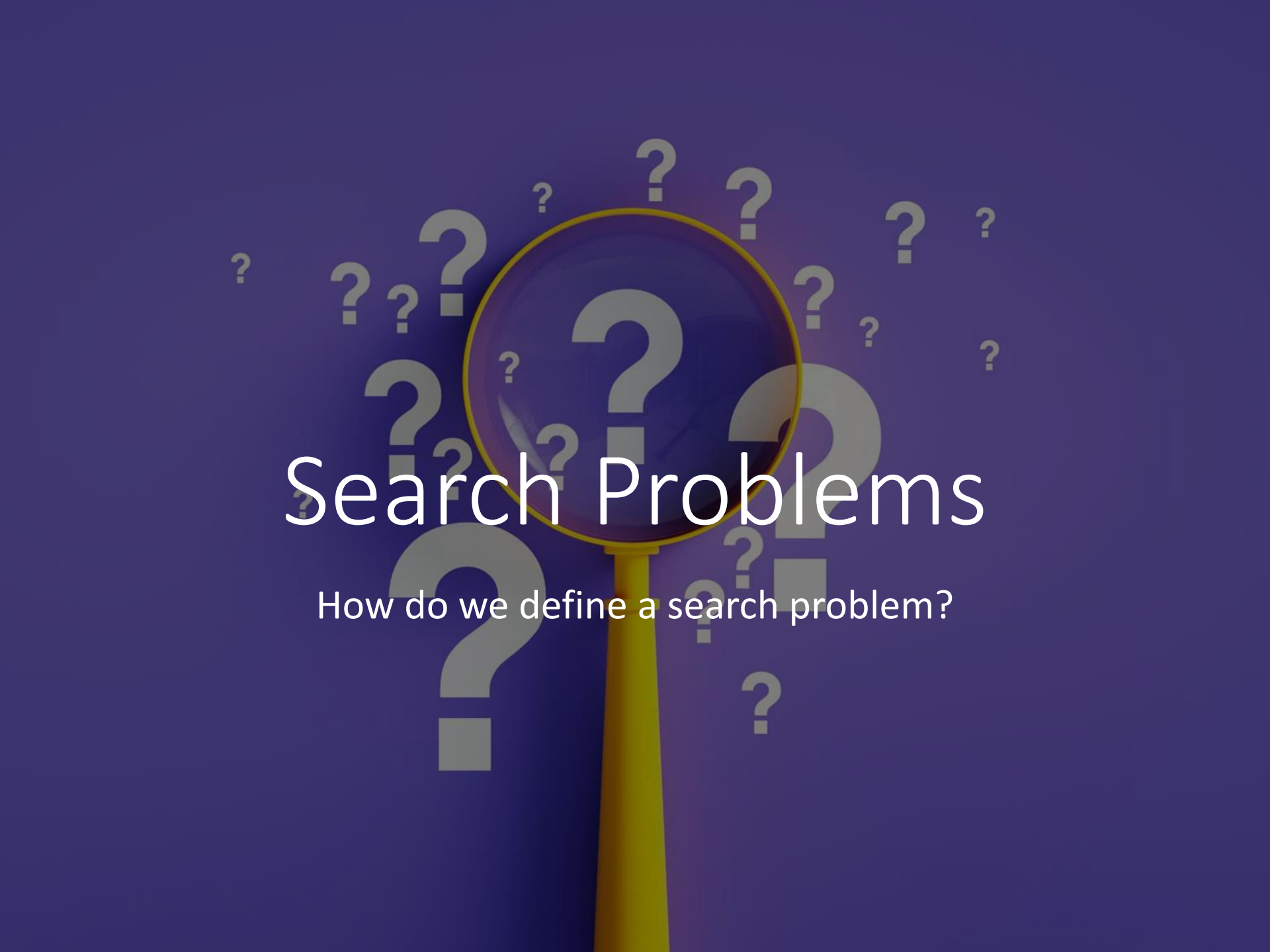


This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.

Online Material

Contents


What are search problems?
Tree search
Search space
Uninformed search
Informed search

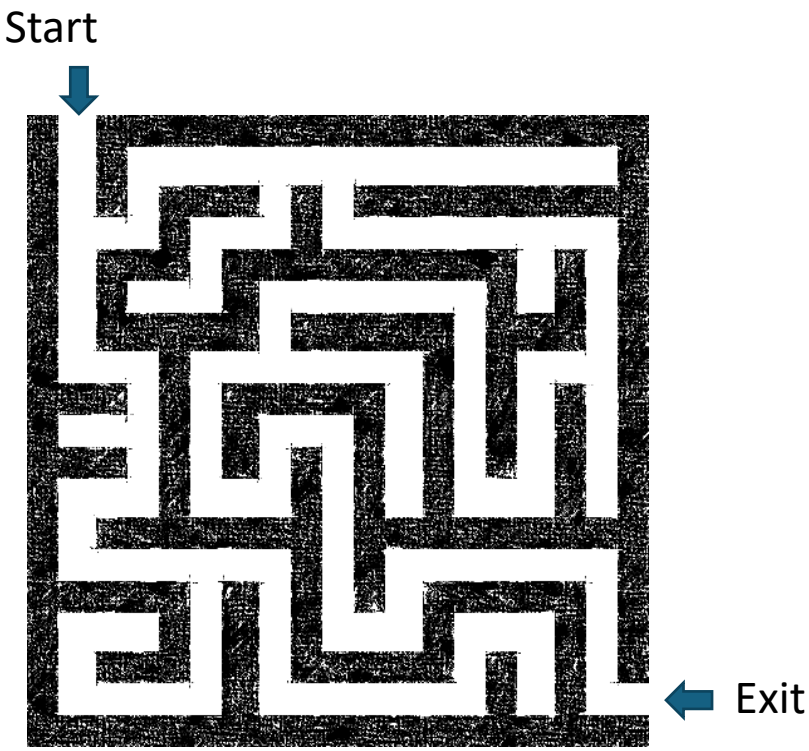


Search Problems

How do we define a search problem?

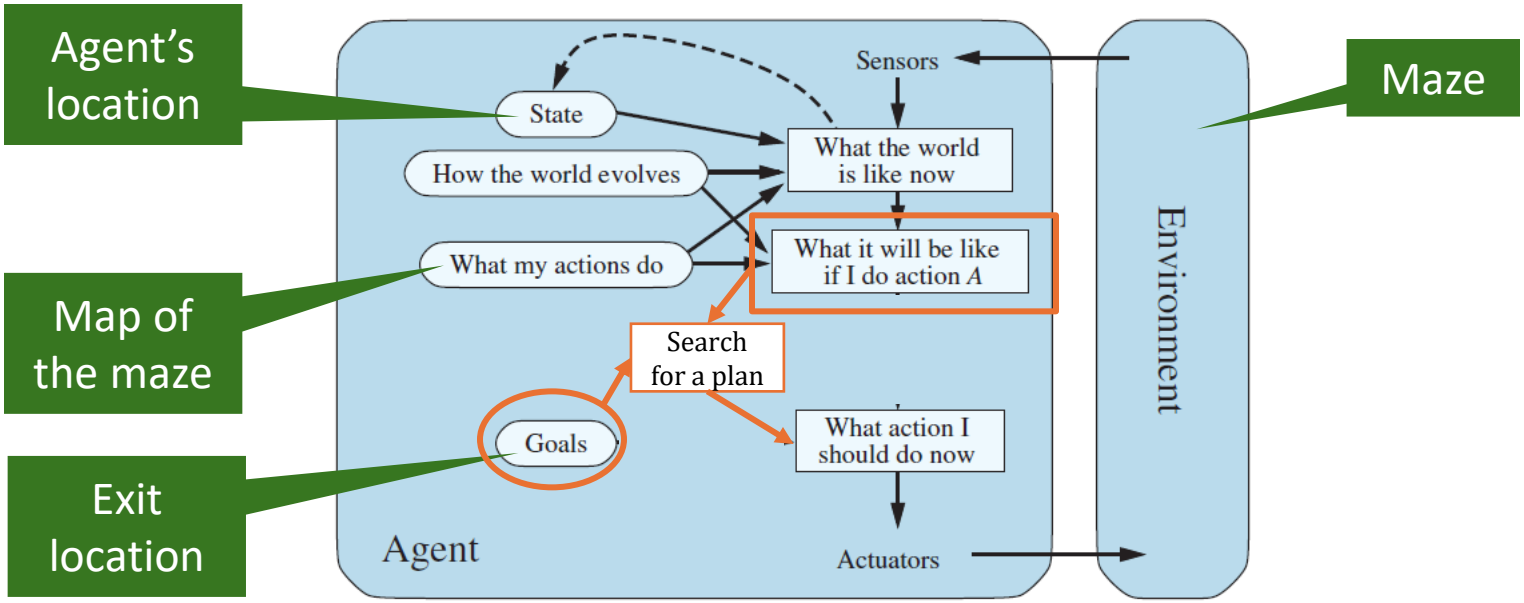
What are Search Problems?

- We will consider the problem of designing **goal-based agents** in **known, fully observable, and deterministic** environments.
- Example environment:



Remember: Planning Agent (Goal-based)

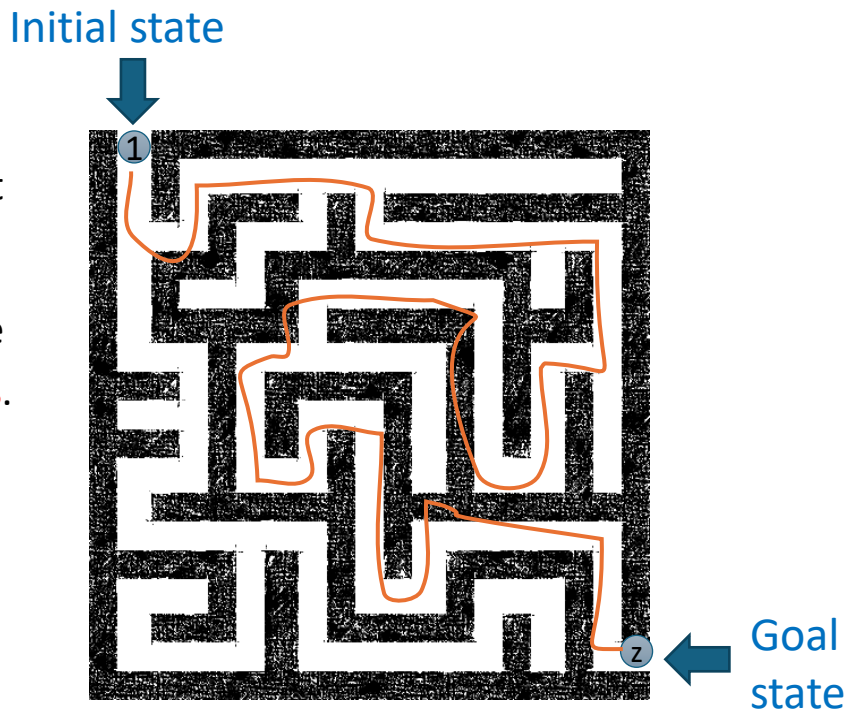
- The agent has the task of reaching a defined **goal state**.
- The performance measure is typically the cost to reach the goal.
- We will discuss a special type of **goal-based agents** called **planning agents,** which use **search algorithms** to plan a sequence of actions that lead to the goal.



$$a_i = \text{argmin}_{a_i \in A}\left[\sum_{t=i}^{T} c_t \,\middle|\, s_T \in S^{goal}\right]$$

Planning for Search Problems

- For now, we consider only a discrete environment using an **atomic state representation** (states are just labeled 1, 2, …).
- The **state space** is the set of all possible states of the environment. Some states are marked as **goal states**.
- The **optimal solution** is the sequence of actions (or equivalently, a sequence of states) that gives the lowest path cost for reaching the goal.

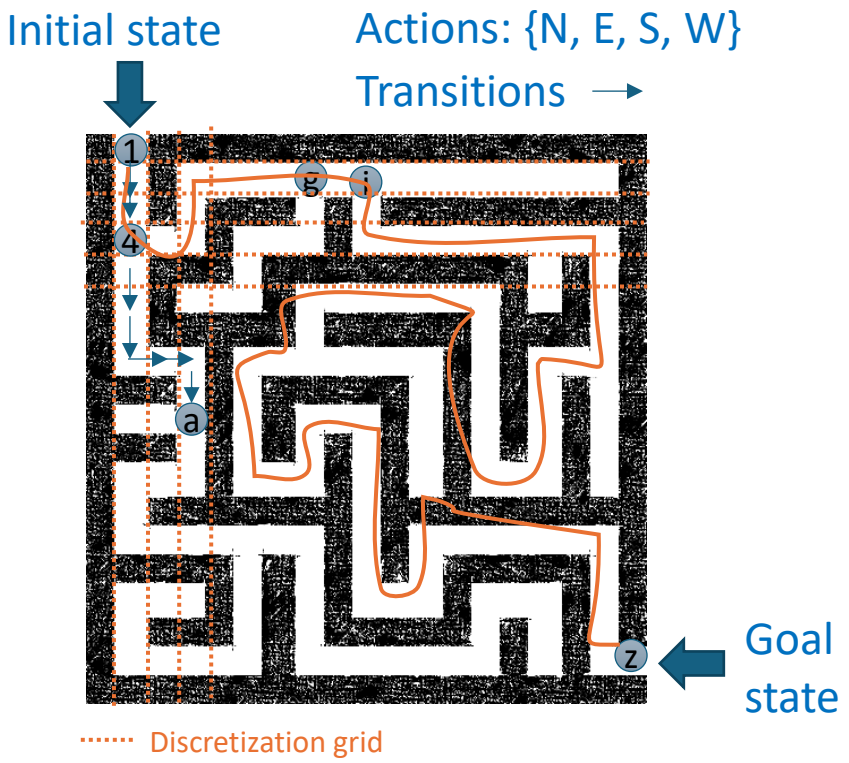


Phases:

1) **Search/Planning**: the process of looking for the **sequence of actions** that reaches a goal state. Requires that the agent knows what happens when it moves!
2) **Execution**: Once the agent begins executing the search solution in a deterministic, known environment, it can ignore its percepts (**open-loop system**).

Definition of a Search Problem

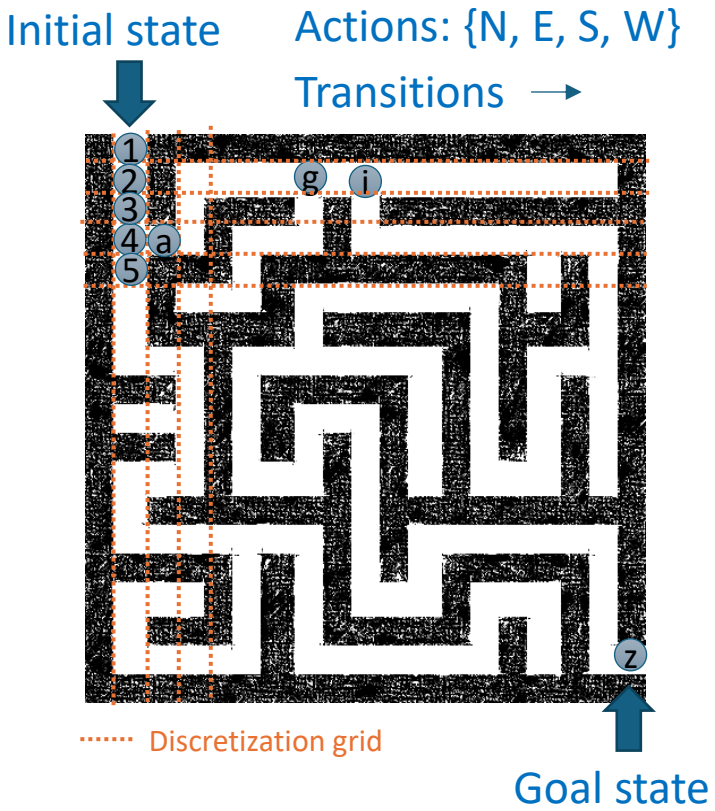
- **Initial state:** state description
- **Actions:** set of possible actions A
- **Transition model:** a function that defines the new state resulting from performing an action in the current state
- **Goal state:** state description
- **Path cost:** the sum of *step costs*



Important: The **state space** is typically too large to be enumerated, or it is continuous. Therefore, the problem is defined by initial state, actions and the transition model and not the set of all possible states.

Transition Function and Available Actions

Original Description



- Definition as an **action schema**:
  $Action(go(dir))$
  PRECOND: no wall in direction dir
  EFFECT: change the agent's location according to dir
- Definition as a **function**:
  $f: S \times A \rightarrow S$ or $s' = result(s, a)$
- A **graph** with states as vertices and actions as edges.
- Function implemented as a **table** representing the state space as a graph.

s	a	s'
1	S	2
2	N	1
2	S	3
…	…	…
4	E	a
4	S	5
4	N	3
…	…	…

- **Available actions** in a state come from the transition function. E.g.,
  $actions(4) = \{E, S, N\}$

Note: Known and deterministic is a property of the transition function!

Example: Romania Vacation

- On vacation in Romania; currently in Arad
- Flight leaves tomorrow from Bucharest

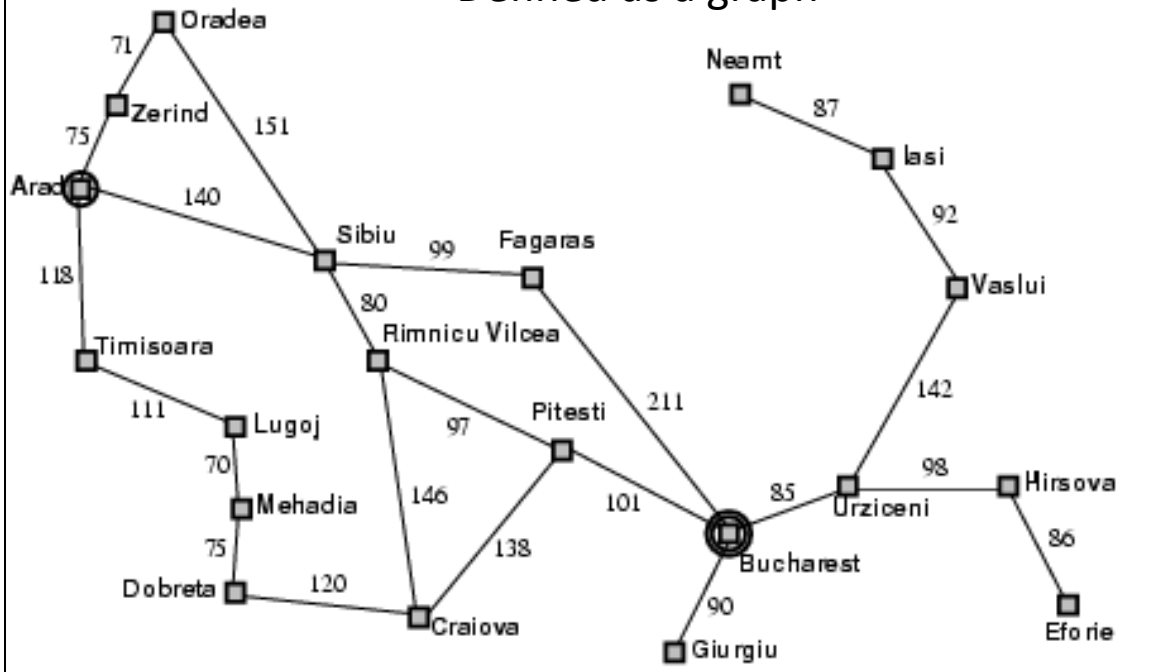
Original Description



- **Initial state:** Arad
- **Actions:** Drive from one city to another.
- **Transition model and states:** If you go from city A to city B, you end up in city B.
- **Goal state:** Bucharest
- **Path cost:** Sum of edge costs.

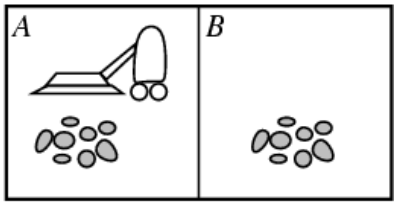
State Space/Transition model

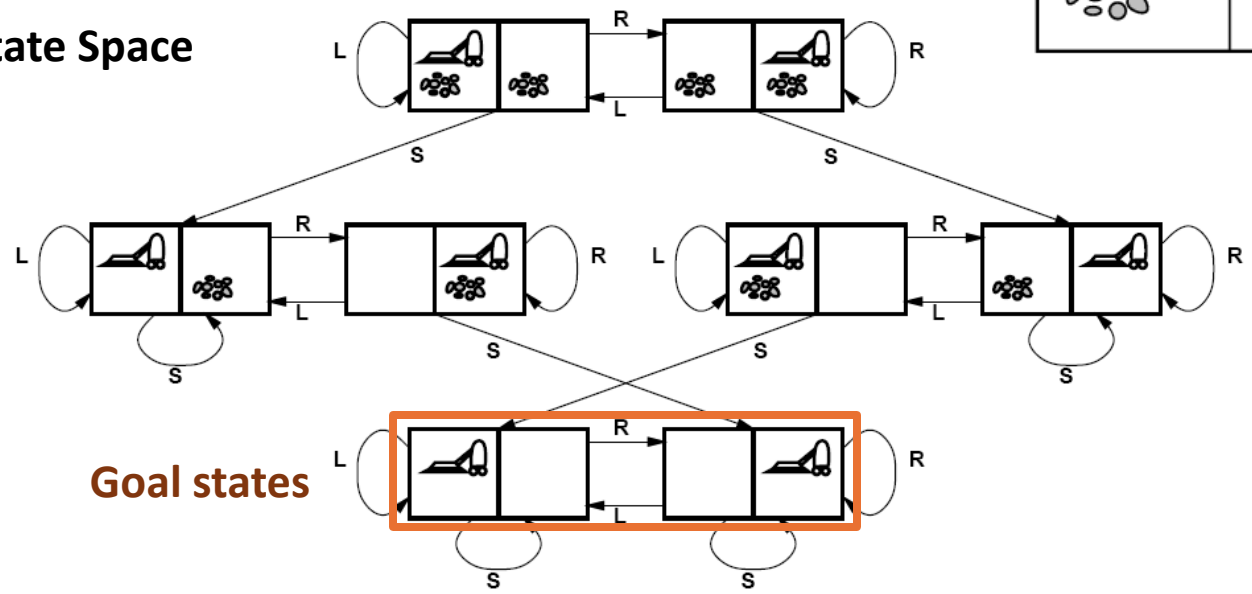
Defined as a graph



Distance in miles

Example: Vacuum world



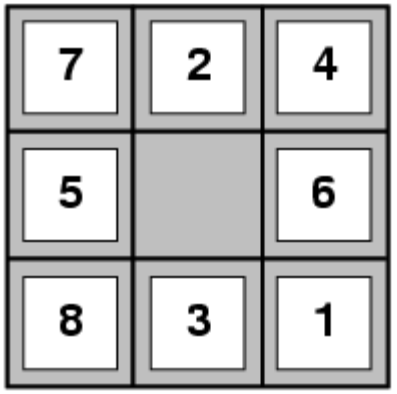


- **Initial State:** Defined by agent location and dirt location.
- **Actions:** Left, right, suck
- **Transition model:** Clean a location or move.
- **Goal state:** All locations are clean.
- **Path cost:** E.g., number if actions

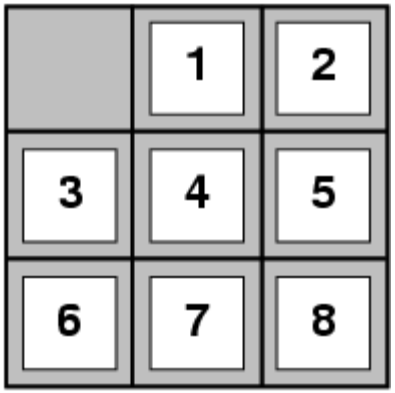
There are 8 possible atomic states of the system.
Why is the number of states for *n* possible locations $n(2^n)$?

Example: Sliding-tile Puzzle

- **Initial State:** A given configuration.
- **Actions:** Move blank left, right, up, down
- **Transition model:** Move a tile
- **Goal state:** Tiles are arranged empty and 1-8 in order
- **Path cost:** 1 per tile move.



Start State



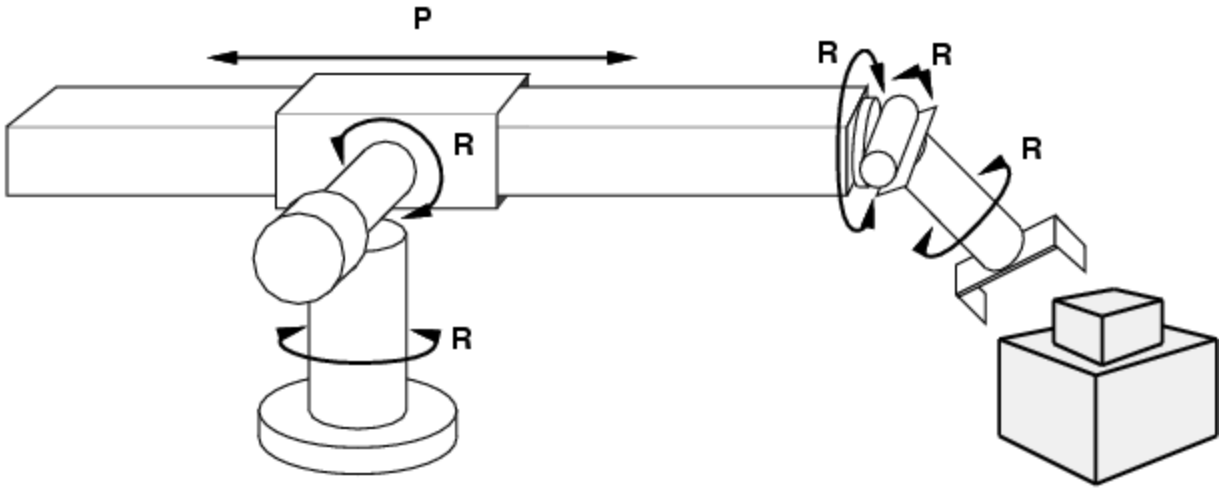
Goal State

State space size

Each state describes the location of each tile (including the empty one). ½ of the permutations are unreachable.

- 8-puzzle: $9!/2 = 181{,}440$ states
- 15-puzzle: $16!/2 \approx 10^{13}$ states
- 24-puzzle: $25!/2 \approx 10^{25}$ states

Example: Robot Motion Planning



- **Initial State:** Current arm position with real-valued coordinates of robot joint angles.
- **Actions:** **Continuous** motions of robot joints.
- **Transition model:** Movement.
- **Goal state:** Desired final configuration (e.g., object is grasped).
- **Path cost:** Time to execute, smoothness of path, etc.